Projet 5 : Monitoring de clusters Kubernetes avec l'IA

Architecture retenue. Cloud Services and Systems, HES-SO Master, Groupe 13.

Ce document décrit une conception de départ. Rien n'est encore déployé ni testé ; les choix seront affinés pendant la mise en place.

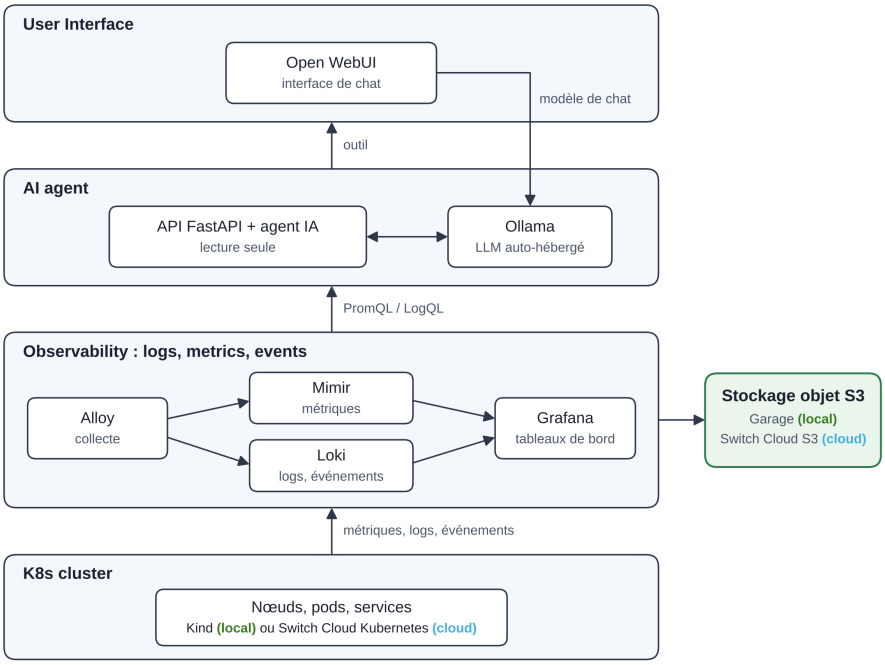
1. Objectif

Un assistant conversationnel qui répond en langage naturel à des questions sur l'état d'un cluster Kubernetes. Il traduit la question en requêtes PromQL et LogQL, récupère métriques, logs et événements, puis rédige une analyse.

Trois cas d'usage : santé globale du cluster, investigation d'un incident, analyse de cause racine. Les traces sont hors périmètre.

2. Architecture

Le schéma reprend les quatre couches de la figure de l'énoncé et y place les composants retenus. Comme dans l'énoncé, les flèches suivent le trajet des données, du cluster vers l'utilisateur. Le même schéma contient l'architecture de test pour un ordinateur **local** et l'architecture finale déployée dans le **cloud**.



Les quatre couches sont déployées dans le même cluster Kubernetes, qui est à la fois la plateforme d'hébergement et le système surveillé. Seul le stockage objet est externe dans la variante cloud.

3. Composants

Composant	Rôle	Origine
Cluster Kubernetes	Héberge toute la solution. Kind en **local**, Switch Cloud Kubernetes dans le **cloud**.	Choix du groupe
Alloy (chart `k8s-monitoring`)	Collecte les métriques, les logs et les événements du cluster.	Imposé
Mimir	Stocke les métriques, interrogeables en PromQL.	Imposé
Loki	Stocke les logs, interrogeables en LogQL.	Imposé
Stockage objet S3	Conserve les données de Mimir et de Loki. Garage en **local**, Switch Cloud S3 dans le **cloud**.	Choix du groupe
Grafana	Tableaux de bord et exploration manuelle.	Imposé
Open WebUI	Interface de chat. Appelle l'API de l'agent comme outil.	Imposé
API FastAPI + agent IA	Image construite par le groupe. Interroge Mimir et Loki, puis rédige l'analyse. LangChain, avec LangGraph pour l'orchestration.	Imposé (FastAPI, LangChain)
Ollama	Sert le modèle de langage à l'agent et à Open WebUI, dans le cluster.	Choix du groupe

Mimir et Loki sont des bases de données auto-hébergées ; le projet n'utilise ni VM gérée directement, ni FaaS, ni SaaS. Le déploiement se fait par charts Helm, pilotés par OpenTofu/Terraform, avec le même module en **local** et dans le **cloud**.

4. Fonctionnement

1. Alloy collecte en continu les données du cluster et les envoie à Mimir et à Loki.
2. L'ingénieur pose sa question dans Open WebUI.
3. Open WebUI (dont le modèle, servi par Ollama, décide d'appeler l'outil) transmet la question à l'API de l'agent.
4. L'agent interroge Mimir et Loki, puis rédige l'analyse avec le LLM.
5. La réponse s'affiche avec les requêtes utilisées, pour que l'ingénieur puisse la vérifier dans Grafana.

5. Choix principaux

- **LLM local (Ollama).** Aucune donnée d'infrastructure n'est envoyée à un service externe. Deux modèles candidats, `qwen3:8b` et `gpt-oss:20b`, à départager par des tests sur les trois cas d'usage.
- **Switch Cloud.** Kubernetes managé dans les data centers suisses de Switch (selon sa documentation), sans VM à administrer. Une alternative de cloud commerciale serait **Exoscale** qui héberge ses datacenters également en Suisse.
- **Kind en local.** Permet de développer et de tester sans coût, avec le même module de déploiement que dans le **cloud**.
- **Garage (S3).** Mimir sans MinIO exige un S3 externe ; Garage est un serveur S3 léger, adapté à un portable.
- **OpenTofu/Terraform + Helm.** Un seul module décrit toute la pile ; entre **local** et **cloud**, seuls le cluster et le S3 changent.
- **Ollama et LangGraph.** Outils de l'article du HUG cité par le sujet : le risque technique est plus faible.
- **Assistant en lecture seule.** Il recommande, il n'agit pas sur le cluster. Il n'a pas accès à l'API Kubernetes.
- **Réponses vérifiables.** L'assistant ne répond qu'à partir des données récupérées et cite ses requêtes, pour limiter les hallucinations.

- **Accès restreint.** Open WebUI demande une authentification ; l'API de l'agent n'est pas exposée hors du cluster.

6. Points ouverts

- Performance du LLM sans GPU dans Kind. Si insuffisante : modèle plus petit, GPU Switch Cloud (quota à demander) ou Ollama sur le Mac.
- Source des alertes pour l'investigation d'incident, à définir.
- Le modèle `qwen3:8b` doit supporter l'appel d'outils avec Open WebUI : à tester en premier.
- Filtrage des données sensibles dans les logs avant envoi au modèle, à concevoir.